



TURBOSTAR.AI

AI Coding environment for native languages

Arjan van de Ven <arjanvandeven@gmail.com>

Native language programmers ...

+-----+

- Compile and link the application and individual files
- Crash the application (or tests) and analyze backtraces and core dumps
- Run the application in a debugger, set breakpoints, single step through code, track and trace data structures
- Analyze performance of the application



Agent CLI Tool Design Matters!

An “N-1” model with a good set of tools *for the task* will outperform a “N” frontier model with a mediocre set of tools



Default behavior: the LLM runs shell commands

This will be our baseline benchmark for better tools



Language Server Protocol & clangd

X-ray vision plumbing for the native language environment

+-----+

- “Where in the code is X”
- “What is in file Y”
- “Autocomplete Z” (type introspection)
- Syntax highlighting (full language parser)

For editors, deep Language Server (LSP) integration makes the difference between a text editor and a development environment.

For Agent CLIs ... we'll let you judge as we go



Compiling (shell)

```
[956/1193] Compiling C++ object test_crash_handler_race.p/tests_unit_test_crash_handler_race.cpp.o
[957/1193] Compiling C++ object test_crash_handler_race.p/src_crash_handler.cpp.o
[958/1193] Compiling C++ object test_debugger_detection.p/src_crash_handler.cpp.o
[959/1193] Compiling C++ object test_debugger_detection.p/tests_unit_test_debugger_detection.cpp.o
[960/1193] Compiling C++ object test_window_scroll.p/tests_unit_test_window_scroll.cpp.o
[961/1193] Compiling C++ object test_gcc_log_parser.p/tests_unit_test_gcc_log_parser.cpp.o
[962/1193] Compiling C++ object test_line.p/tests_unit_test_line.cpp.o
[963/1193] Compiling C++ object test_image_tools.p/tests_unit_test_image_tools.cpp.o
[964/1193] Compiling C++ object test_document.p/tests_unit_test_document.cpp.o
[965/1193] Compiling C++ object test_hex_highlighter.p/tests_unit_test_hex_highlighter.cpp.o
[966/1193] Compiling C++ object
test_crashdump_clear.p/src_tools_crashdump_clear_crashdump_clear_entry.cpp.o
[967/1193] Compiling C++ object test_crashdump_clear.p/tests_unit_test_crashdump_clear.cpp.o
[968/1193] Compiling C++ object test_model_server.p/tests_unit_test_model_server.cpp.o
[969/1193] Compiling C++ object test_session_manager.p/tests_unit_test_session_manager.cpp.o
[970/1193] Compiling C++ object test_copilot_models.p/tests_unit_test_copilot_models.cpp.o
[971/1193] Compiling C++ object test_tool_status.p/tests_unit_test_tool_status.cpp.o
[972/1193] Compiling C++ object test_sqlite_validation.p/tests_unit_test_sqlite_validation.cpp.o
[973/1193] Compiling C++ object test_project_layout.p/tests_unit_test_project_layout.cpp.o
[974/1193] Compiling C++ object test_available_tests.p/tests_unit_test_available_tests.cpp.o
[975/1193] Compiling C++ object test_codereview_manager.p/tests_unit_test_codereview_manager.cpp.o
[976/1193] Compiling C++ object test_live_llm.p/tests_unit_test_live_llm.cpp.o
[977/1193] Compiling C++ object test_httplib_transport.p/tests_unit_test_httplib_transport.cpp.o
```



Compiling (shell)

```
[956/1193] Compiling C++ object test_crash_handler_race.p/tests_unit_test_crash_handler_race.cpp.o
[957/1193] Compiling C++ object test_crash_handler_race.p/src_crash_handler.cpp.o
[958/1193] Compiling C++ object test_debugger_detection.p/src_crash_handler.cpp.o
[959/1193] Compiling C++ object test_debugger_detection.p/tests_unit_test_debugger_detection.cpp.o
[960/1193] Compiling C++ object test_window_scroll.p/tests_unit_test_window_scroll.cpp.o
[961/1193] Compiling C++ object test_gcc_log_parser.p/tests_unit_test_gcc_log_parser.cpp.o
[962/1193] Compiling C++ object test_line.p/tests_unit_test_line.cpp.o
[963/1193] Compiling C++ object test_image_tools.p/tests_unit_test_image_tools.cpp.o
[964/1193] Compiling C++ object test_document.p/tests_unit_test_document.cpp.o
[965/1193] Compiling C++ object test_hex_highlighter.p/tests_unit_test_hex_highlighter.cpp.o
[966/1193] Compiling C++ object test_crashdump_clear.p/src_tools_crashdump_clear_crashdump_clear_entry.cpp.o
[967/1193] Compiling C++ object test_crashdump_clear.p/tests_unit_test_crashdump_clear.cpp.o
[968/1193] Compiling C++ object test_model_server.p/tests_unit_test_model_server.cpp.o
[969/1193] Compiling C++ object test_session_manager.p/tests_unit_test_session_manager.cpp.o
[970/1193] Compiling C++ object test_copilot_models.p/tests_unit_test_copilot_models.cpp.o
[971/1193] Compiling C++ object test_tool_status.p/tests_unit_test_tool_status.cpp.o
[972/1193] Compiling C++ object test_sqlite_validation.p/tests_unit_test_sqlite_validation.cpp.o
[973/1193] Compiling C++ object test_project_layout.p/tests_unit_test_project_layout.cpp.o
[974/1193] Compiling C++ object test_available_tests.p/tests_unit_test_available_tests.cpp.o
[975/1193] Compiling C++ object test_codereview_manager.p/tests_unit_test_codereview_manager.cpp.o
[976/1193] Compiling C++ object test_live_llm.p/tests_unit_test_live_llm.cpp.o
[977/1193] Compiling C++ object test_httpplib_transport.p/tests_unit_test_httpplib_transport.cpp.o
```

1200 lines into context!



Compiling (fs_compile_project)

```
ninja: Entering directory `/home/arjan/git/turbostar2/build'
[1021/1193] Compiling C++ object turbostar.p/src_main.cpp.o
../src/main.cpp: In function 'std::string get_home_dir()':
../src/main.cpp:33:13: warning: unused variable 'foo' [-Wunused-variable]
[symbol: get_home_dir (lines 31-38)]
    33 |         int foo;
        |         ^~~
[1193/1193] Linking target test_editor_tool_family
INFO: autodetecting backend as ninja
INFO: calculating backend command to run: /usr/bin/ninja -C
```



Compiling (fs_compile_project)

```
ninja: Entering directory `/home/arjan/git/turbostar2/build'
[1021/1193] Compiling C++ object turbostar.p/src_main.cpp.o
../src/main.cpp: In function 'std::string get_home_dir()':
../src/main.cpp:33:13: warning: unused variable 'foo' [-Wunused-variable]
[symbol: get_home_dir (lines 31-38)]
    33 |         int foo;
        |         ^~~
[1193/1193] Linking target test_editor_tool_family
INFO: autodetecting backend as ninja
INFO: calculating backend command to run: /usr/bin/ninja -C
```

10 lines into context!



Compiling (fs_compile_project)

```
ninja: Entering directory `/home/arjan/git/turbostar2/build'
[1021/1193] Compiling C++ object turbostar.p/src_main.cpp.o
../src/main.cpp: In function 'std::string get_home_dir()':
../src/main.cpp:33:13: warning: unused variable 'foo' [-Wunused-variable]
[symbol: get_home_dir (lines 31-38)]
33 |         int foo;
    |         ^~~
[1193/1193] Linking target test_editor_tool_family
INFO: autodetecting backend as ninja
INFO: calculating backend command to run: /usr/bin/ninja -C
```

← **LSP in action**



Compiling (fs_compile_project)

```
Compilation
[1181/1193] Linking target test_list_code_review_items
[1182/1193] Linking target test_codereview_line_tracking
[1183/1193] Linking target test_create_code_review_item
[1184/1193] Linking target test_codereview_manager
[1185/1193] Linking target test_run_config
[1186/1193] Linking target test_security_review_with_agent
[1187/1193] Linking target test_confirm_code_review_item
[1188/1193] Linking target test_code_review_window
[1189/1193] Linking target test_get_code_review_item
[1190/1193] Linking target test_resolve_code_review_item
[1191/1193] Linking target test_live_llm
[1192/1193] Linking target test_perform_code_review
[1193/1193] Linking target test_editor_tool_family
INFO: autodetecting backend as ninja
INFO: calculating backend command to run: /usr/bin/ninja -C /home/arjan/git/turbostar2/build
```

Human user view:

Full ansi (color) compatible terminal emulator in a “virtual window” in the scrolling agent history showing the whole compilation process streaming live



Native code crashes due to bugs

Crash information by priority

1. The crash location (file:line)
2. The assertion message
3. Backtrace (with symbols and file:line resolved)
4. Core dump file + gdb

"My agent already does all of this today!"

LLMs can solve many issues with just minimal information (#1)....
.... or jump to conclusions too quickly on slightly more complex issues

Stating the obvious

Each additional layer of information improves confidence in the root cause analysis
Each additional layer of information allows the LLM to solve even more complex bugs

Least Friction Principle: LLMs will use information that is either easy to get, or to which they are steered



Run application example (wait_for_app)

Tool call:

```
{
  "arguments": {
    "run_id": 1001,
    "timeout_sec": 30,
    "type": "ended"
  },
  "name": "agent_wait_for_app"
}
```

Tool call return:

```
{
  "crash_notification": "\n\n
CRASH DETECTED: Application crashed (Crash ID: 1028928).\n
Summary: assertion fail at ../main.cpp:8 `c != NULL` in void foo(char*)()\n
Please use 'crashdump_get_info' with crash_id '1028928' to investigate stack trace and
details.",
  "is_alive": false,
  "status": "ended"
}
```



```
## Crash Report
```

```
### Failed Assertion  
```
```

```
Assertion: c != NULL
```

```
File: ../main.cpp
```

```
Line: 8
```

```
Function: void foo(char*)
```

```
```
```

```
### Info
```

```
```
```

```
Signal: 6
```

```
Executable: /home/arjan/git/turbotest9a/build/testcase
```

```
CrashCookie: run_1001
```

```
```
```

```
(continues on next slide)
```



Backtrace

Frame	Address	Function	Location	Note
0-4		`<crash handling frames>`	<turbocatch>	crash handling
5		`__pthread_kill_implementation (<cut to fit on slide>)`	<libc>	
6		`__pthread_kill_internal (signo=6)`	<libc>	
7		`__GI_raise (sig=sig@entry=6)`	<libc>	
8		`__GI_abort ()`	<libc>	
9		`__assert_fail_base (function=0x5591de09a004 "void foo(char*)")`	<libc>	
10		`__assert_fail (<cut to fit on slide>)`	<turbocatch>	
11	`0x5591de099191`	`foo (c=0x0)`	main.cpp:8	
12	`0x5591de0991ea`	`main (argc=1, argv=0x7ffe7cd14698)`	main.cpp:16	

Codemap Summary

File	Symbol	Start Line	End Line	Lines
`main.cpp`	`foo`	5	11	7
`main.cpp`	`main`	12	18	7

CoreDump Debugging

A coredump is available for this crash. You can launch a GDB session to debug this coredump by calling the 'agent_debug_coredump' tool:

```json

```
{
 "name": "agent_debug_coredump",
 "arguments": {
 "crash_id": "1013171"
 }
}
```



Core was generated by `/home/arjan/git/turbotest9a/build/testcase'.

```
#0 __internal_syscall_cancel (a1=<optimized out>, a2=<optimized out>, a3=<optimized out>, a4=<optimized out>, a5=a5@entry=0, a6=a6@entry=0, nr=61)
 at ./nptl/cancellation.c:44
```

⚠ warning: 44 ./nptl/cancellation.c: No such file or directory

(gdb) bt

```
#0 __internal_syscall_cancel (a1=<optimized out>, a2=<optimized out>, a3=<optimized out>, a4=<optimized out>, a5=a5@entry=0, a6=a6@entry=0, nr=61)
 at ./nptl/cancellation.c:44
#1 0x00007fe01549bccc in __syscall_cancel (a1=<optimized out>, a2=<optimized out>, a3=<optimized out>, a4=<optimized out>, a5=a5@entry=0, a6=a6@entry=0, nr=61)
 at ./nptl/cancellation.c:75
#2 0x00007fe015507ae7 in __GI___wait4 (pid=<optimized out>, stat_loc=<optimized out>, options=<optimized out>, usage=<optimized out>)
 at ../sysdeps/unix/sysv/linux/wait4.c:30
#3 0x00007fe015905076 in turbocatch_handle_signal (sig=6, info=0x7fff7b3225f0, ucontext=0x7fff7b3224c0) at ../src/crash_catcher/crash_catcher.c:337
#4 <signal handler called>
#5 __pthread_kill_implementation (threadid=<optimized out>, signo=signo@entry=6, no_tid=no_tid@entry=0) at ./nptl/pthread_kill.c:44
#6 0x00007fe0154a129f in __pthread_kill_internal (threadid=<optimized out>, signo=6) at ./nptl/pthread_kill.c:89
#7 0x00007fe01544a842 in __GI_raise (sig=sig@entry=6) at ../sysdeps/posix/raise.c:26
#8 0x00007fe0154324b2 in __GI_abort () at ./stdlib/abort.c:77
#9 0x00007fe015432424 in __assert_fail_base (fmt=<optimized out>, assertion=<optimized out>, file=<optimized out>, line=<optimized out>,
 function=0x56023936a004 "void foo(char*)") at ./assert/assert.c:118
#10 0x00007fe0159054e4 in __assert_fail (assertion=0x56023936a020 "c != NULL", file=0x56023936a014 "../main.cpp", line=8, function=0x56023936a004 "void foo(char*)")
 at ../src/crash_catcher/crash_catcher.c:420
#11 0x0000560239369191 in foo (c=0x0) at ../main.cpp:8
#12 0x00005602393691ea in main (argc=1, argv=0x7fff7b323da8) at ../main.cpp:16
```

(gdb) frame 11

```
#11 0x0000560239369191 in foo (c=0x0) at ../main.cpp:8
```

```
8 assert(c != NULL);
```

(gdb) info locals

```
buffer = "\002V\000"
```

```
__PRETTY_FUNCTION__ = "void foo(char*)"
```

(gdb) info args

```
c = 0x0
```

(gdb)

[Capture: ON (^G to escape)]

## The agent drives GDB on the coredump

### Tool call:

```
{ "arguments": {
 "data": "frame 11\ninfo locals\ninfo args\n",
 "output": true,
 "run_id": 2002
},
 "name": "agent_write_to_run"
}
```

### Tool call result:

```
<agent_write_to_run_result>
frame 11
#11 0x0000560239369191 in foo (c=0x0) at ../main.cpp:8
8 assert(c != NULL);
(gdb) info locals
buffer = "\002V\000"
__PRETTY_FUNCTION__ = "void foo(char*)"
(gdb) info args
c = 0x0
(gdb)
</agent_write_to_run_result>
```



# Agents Debugging the software

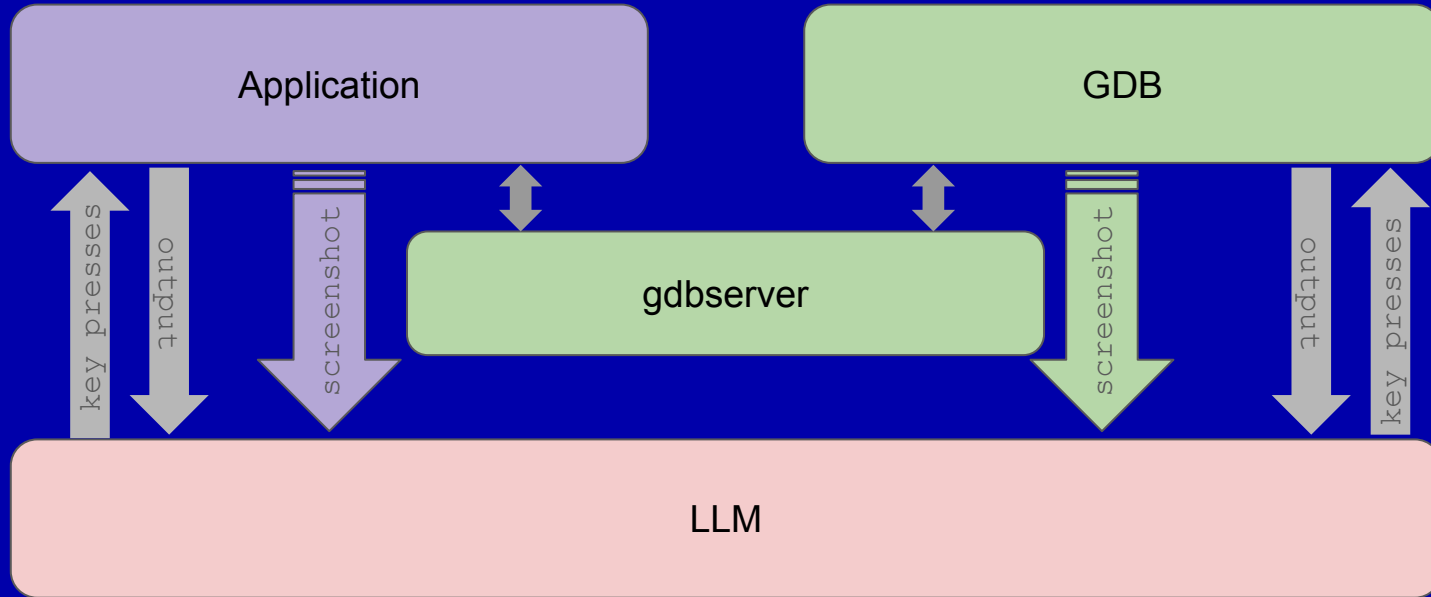
---

- Many non-native languages fall into the “add a print” world of debugging
- With native languages we can do better!
- ... but in reality, the current models seem to be trained to add print statements as the easiest option (until you give them good tools)



# Agents Debugging the software

---



File Edit Search Tools Run Options Git Agent Window Help

Run Output

[≡]==1002=

Hello, world!

[Capture: ON (^G to escape)]

Debugger (GDB)

[≡]==1003=

Continuing.

Reading /usr/lib/x86\_64-linux-gnu/libc.so.6 from remote target...

Program received signal SIGABRT, Aborted.

\_\_pthread\_kill\_implementation (threadid=<optimized out>, signo=signo@entry=6, no\_tid=no\_tid@entry=0) at ./nptl/pthread\_kill.c:44

⚠ warning: 44 ./nptl/pthread\_kill.c: No such file or directory

(gdb)

[Break] [Step] [Next] [Cont] [Quit]

[Capture: ON (^G to escape)]

F1 Help

# Performance analysis

---

- Performance matters for native applications
  - we'd be using python/javascript otherwise
- LLMs "kind of" know how to drive Linux perf or vtune
  - Intel and others publish Skills for this
- But "perf" is not a great fit for LLMs



# Linux Perf tool and LLMs: not a natural fit

---

- ▶ **LLMs “think” in terms of functions and code lines**

perf operates primarily on assembly instructions (but can aggregate into functions)

- ▶ **Perf output parsing can be messy**

Perf is designed for humans to read. The output varies often by version. Some perf versions have machine readable output, but availability varies by distro

- ▶ **Perf generates a LOT of data that needs to be reduced to usable information**

High token/context cost



# What native perf integration looks like

## Tool call:

```
{
 "arguments": {
 "collect_performance": true,
 "wait_for_time": 60
 },
 "name": "run_executable"
}
```

## Tool call result:

```
{
 "app_run_id": 1001,
 "is_alive": false,
 "profile_notification": "Performance profile data is available (5082 samples
collected). Call agent_get_profile_summary(run_id=\"1001\") to view top
bottleneck functions.",
 "status": "ended"
}
```



✓ Starting application - Started run\_id 1001 (status: ended)

The application ran successfully and collected 5,082 CPU samples. Let me retrieve the profile summary to identify the bottlenecks.

✓ Retrieving performance profile summary - Profile summary retrieved.

The performance data collection worked successfully. Here's a summary of what I found:

## ✓ Performance collection successful

I ran your `testcase` application with CPU performance profiling enabled. The application executed and the profiler captured 5,082 samples, which I've now analyzed.

### Top performance bottlenecks

The overwhelming majority of CPU time is spent in the prime-checking functions:

| Function                         | CPU samples | % of total | Hot line                  |
|----------------------------------|-------------|------------|---------------------------|
| <code>is_prime_vA</code> (naive) | 3,277       | 64.5%      | <code>prime.cpp:6</code>  |
| <code>is_prime_vB</code> (n/2)   | 1,750       | 34.4%      | <code>prime.cpp:16</code> |
| <code>is_prime_vC</code> (sqrt)  | 29          | 0.57%      | <code>prime.cpp:27</code> |
| <code>is_prime_vD</code> (6k±1)  | 17          | 0.33%      | <code>prime.cpp:38</code> |

### Where the time actually goes (hot lines)

Zooming into the line-level attribution:

▼ Follow

[ 8.5k / 250k ]

> can you do the toolcall to get a detailed report for `is_prime_vA` ?

✓ Retrieving performance profile details - Retrieved profile details (7 line entries)

Here's the detailed line-level performance report for `is_prime_vA`:

## Detailed profile: `is_prime_vA` (prime.cpp)

| Line | Source                                                       | Samples | % of file | % of total run |
|------|--------------------------------------------------------------|---------|-----------|----------------|
| 3    | <code>bool is_prime_vA(int number) {</code> (function entry) | 1,797   | 35.4%     | 35.4%          |
| 5    | <code>for (int i = 2; i &lt; number; i++) {</code> (loop)    | 224     | 4.4%      | 4.4%           |
| 6    | <code>if (number % i == 0) return false;</code> (modulo)     | 3,051   | 60.1%     | 60.0%          |
| 9    | <code>}</code> (return)                                      | 1       | ~0%       | ~0%            |

Total attributed: 5,073 samples (99.8% of the 5,082 captured).

### Key takeaways

- Line 6 is the hot spot – the inner `number % i` modulo-division check accounts for ~60% of the entire program's CPU time. This is the core of the naive algorithm's cost.
- Lines 3 + 5 together account for the remaining ~40% – the function entry (call/return overhead attributed to the entry) and the loop-bounds computation.
- The `%` (modulo) operation on line 6 is the single most expensive instruction, which makes sense given that `is_prime_vA` linearly scans every divisor from 2 to `n-1` for each number up to 25,000.

This strongly confirms the algorithmic bottleneck: the naive trial-division loop on line 6 is where you'd want to focus optimization efforts (which is exactly why versions C and D improve on it).

Want me to pull the same detailed report for any of the other functions (`is_prime_vB`, `_vC`, or `_vD`)?

[ 8.5k / 250k ]

## Bonus features

Features that also help other languages



# fs\_file\_codemap() - X-ray vision

### Codemap for `src/main.cpp` (2 symbols):

| Function       | Start Line | End Line |
|----------------|------------|----------|
| -----          | -----      | -----    |
| `get_home_dir` | 32         | 38       |
| `main`         | 40         | 473      |



# Reading lines of a file - Proactive Context Injection

## Over-reading to match function boundaries

The LLM asks to read up to line 100, but the function goes from line 90 to 106

Answer: Return the data all the way to line 106

## Automatic mini code map for the file

Gives the LLM the locations of top functions in this same file

Sorted by proximity, past history of searches, what was shown before, ...

## Header <-> Code link code map

When reading a header file, provide a mini code map for the matching code file

## Code map for functions called from the snippet

The LLM is likely to want to investigate (read) functions called from the code, so provide the locations of top functions

## C++ Class summary

The LLM gets the datatypes of used class variables and methods to avoid making assumptions which leads to type bugs



# Over-reading snaps to function boundaries

```
{
 "arguments": {
 "path": "src/document.cpp",
 "start_line": 410,
 "end_line": 415
 },
 "name": "fs_read_lines"
}
```

```


```

Code for lines 410 - 418 of src/document.cpp (total 872 lines):

```
```cpp  
410: int document::restore_cursor_state(const cursor_state &state)  
411: {  
412:     std::unique_lock lock(mutex_);  
413:     int delta_y = restore_cursor_state_unlocked(state);  
414:     lock.unlock();  
415:     notify_cursor_changed();  
416:     request_redraw();  
417:     return delta_y;  
418: }  
```
```



# Automatic mini code maps

### Codemap for `src/editor.cpp` (Top 10 of 50 symbols):

| Function                    | Start Line | End Line |
|-----------------------------|------------|----------|
| -----                       | -----      | :-----:  |
| `editor`                    | 46         | 1834     |
| `  ::editor`                | 46         | 177      |
| `  ::new_window`            | 179        | 202      |
| `  ::open_file_as_text`     | 204        | 215      |
| `  ::open_file_as_binary`   | 217        | 226      |
| `  ::open_prompt_in_editor` | 228        | 256      |
| `  ::new_diff_window`       | 258        | 267      |
| `  ::update_window_layout`  | 269        | 317      |
| `  ::new_agent_window`      | 319        | 344      |
| `  ::set_focus`             | 1024       | 1035     |

\*... [40 other symbols omitted (use fs\_file\_codemap if full symbol table is needed)]\*

### Codemap for `src/fs\_utils.cpp` (Called Dependency):

| Function        | Start Line | End Line |
|-----------------|------------|----------|
| :-----          | :-----:    | :-----:  |
| `get_file_type` | 236        | 277      |



# C++ Class summary: instant access to data types

```
Class Context: `document` (extracted from `src/document.h` for referenced members):
```cpp  
// Referenced member variables:  
269:     mutable std::shared_mutex mutex_  
  
// Referenced member functions:  
134:     int restore_cursor_state(const cursor_state &state);  
139:     void request_redraw() const;  
178:     void notify_cursor_changed() const;  
```
```



# Editing files

## Real-time brace balance checks

Give the LLM immediate feedback if a file goes from balanced {}'s to unbalanced braces (typical LLM edit glitch)

## Instant compile check

If the edit caused the file to stop compiling, remember the number of the edit and point to it at compile time as the introducing edit

## Function-scoped edits

The LLM can pass in the name of the function being edited, greatly reducing the probability of misaligned edits

## Post-edit return

Return the edited code so that the LLM can see if the edit achieved the intent



## Appendix

When you thought life was easy



# Coredumps on Linux are hard (for automation)

## containers

Coredumps inside a container go to "somewhere" outside the container.  
And then there is ChromeOS...

## coredumpctl - or not

In theory the answer. In practice, highly variable between distros and container environments. Debian? Nope.

## ulimit and other permissions

Too often there are super conservative defaults or LSMs interfering

## compression

Coredumps may end up being compressed on disk - or not, depending on the distro

